資料結構

Data Structure

Lab05

姓名：李品融

學號：113310217

Lab05-Q1
這題在練習 Stack 的相關應用，利用隨機洗牌的方式將牌組壓入堆疊，並利用後進先出的特性依序發牌。

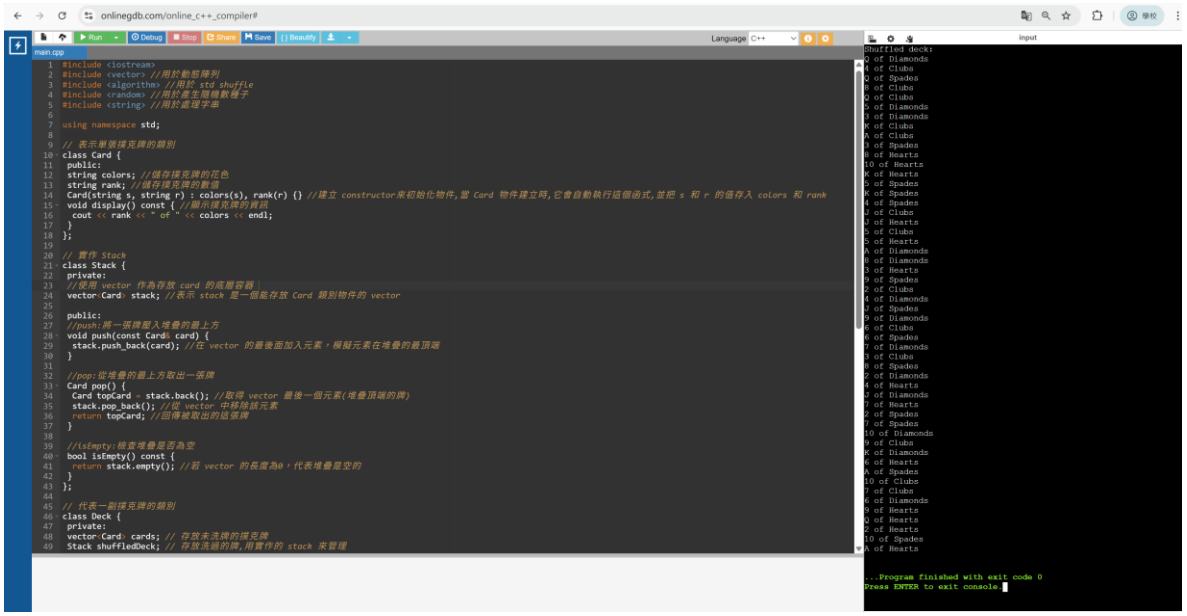
Code

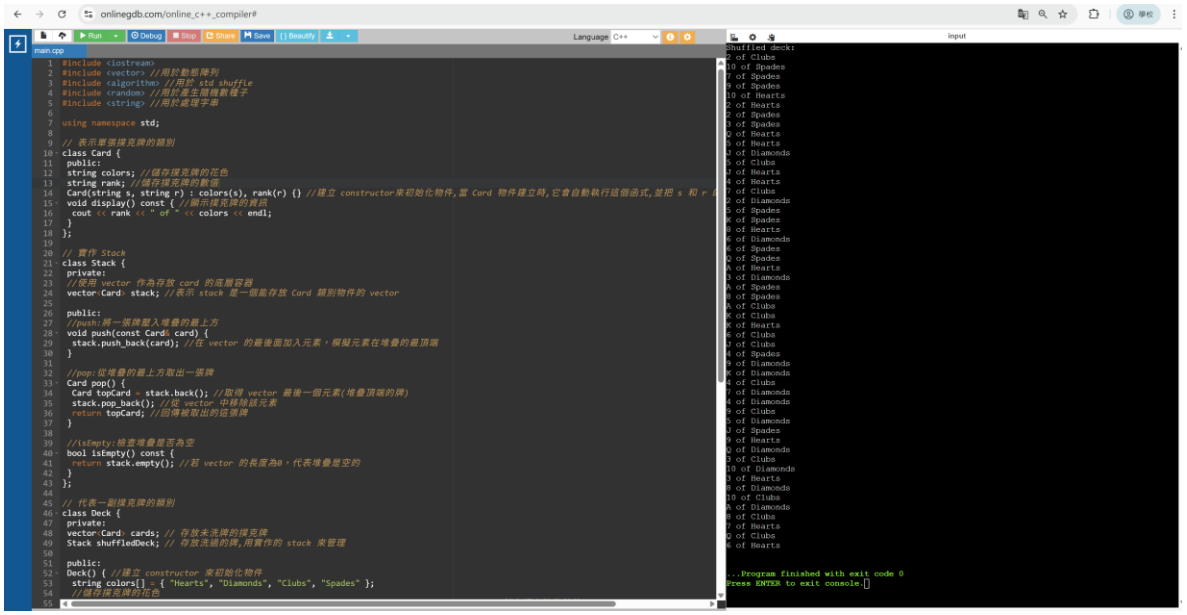
```
#include <iostream>
#include <vector> //用於動態陣列
#include <algorithm> //用於 std shuffle
#include <random> //用於產生隨機數種子
#include <string> //用於處理字串

using namespace std;

// 表示單張撲克牌的類別
class Card {
 public:
 string colors; //儲存撲克牌的花色
 string rank; //儲存撲克牌的數值
 Card(string s, string r) : colors(s), rank(r) {} //建立 constructor 來初始化物件,當 Card 物件建立時,它會自動執行這個函式,並把 s 和 r 的值存入 colors 和 rank
 void display() const { //顯示撲克牌的資訊
  cout << rank << " of " << colors << endl;
 }
};

// 實作 Stack
class Stack {
 private:
 //使用 vector 作為存放 card 的底層容器
 vector<Card> stack; //表示 stack 是一個能存放 Card 類別物件的 vector

 public:
 //push:將一張牌壓入堆疊的最上方
 void push(const Card& card) {
  stack.push_back(card); //在 vector 的最後面加入元素，模擬元素在堆疊的最頂端
 }
```

```
 //pop:從堆疊的最上方取出一張牌
 Card pop() {
  Card topCard = stack.back(); //取得 vector 最後一個元素(堆疊頂端的牌)
  stack.pop_back(); //從 vector 中移除該元素
  return topCard; //回傳被取出的這張牌
 }

 //isEmpty:檢查堆疊是否為空
 bool isEmpty() const {
  return stack.empty(); //若 vector 的長度為 0，代表堆疊是空的
 }
};

// 代表一副撲克牌的類別
class Deck {
 private:
 vector<Card> cards; // 存放未洗牌的撲克牌
 Stack shuffledDeck; // 存放洗過的牌,用實作的 stack 來管理

 public:
 Deck() { //建立 constructor 來初始化物件
  string colors[] = { "Hearts", "Diamonds", "Clubs", "Spades" };
  //儲存撲克牌的花色
  string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9","10", "J", "Q",
"K", "A" }; //儲存撲克牌的數值

  //利用迴圈將 52 張牌加入 cards 這個 vector 裡面
  for (int i = 0; i < 4; i++) { // 四種花色
   for (int j = 0; j < 13; j++) { // 13 種點數
    cards.push_back(Card(colors[i], ranks[j]));
   }
  }
 }
```

```
 //洗牌(Hint:使用 shuffle 函數)
 void shuffleDeck() {
  random_device rd; //取得硬體的隨機數種子
  mt19937 g(rd()); //使用演算法產生隨機數
  shuffle(cards.begin(), cards.end(), g); //使用 shuffle 函式打亂 cards vector 的順序

  for(int i = 0; i < cards.size(); i++) { //將洗亂的牌送進 Stack 裡面
   shuffledDeck.push(cards[i]);
  }
 }
 //發牌，從 Stack 取出所有牌
 void drawAllCards() {
  while(!shuffledDeck.isEmpty()){ //一直抽牌直到 Stack 裡面沒有牌
   Card c=shuffledDeck.pop();
   c.display(); //將抽到的牌顯示出來
  }
 }
};


int main() {
 Deck deck; //建立 deck 產生 52 張撲克牌
 deck.shuffleDeck(); //進行洗牌並放入堆疊
 cout << "Shuffled deck:" << endl;
 deck.drawAllCards(); //依序取出堆疊內的牌並顯示
 return 0;
}
```

Result


onlinegdb.com/online_c++_compiler#
Run
Debug
Stop
Share
Save
{ } Beautify
Language C++
main.cpp
#include <iostream>
#include <vector> //用於動態陣列
#include <algorithm> //用於 std shuffle
#include <random> //用於產生隨機數種子
#include <string> //用於處理字串
using namespace std;
// 表示單張撲克牌的類別
class Card {
public:
string colors; //儲存撲克牌的花色
string rank; //儲存撲克牌的數值
Card(string s, string r) : colors(s), rank(r) {} //建立 constructor來初始化物件,當 Card 物件建立時,它會自動執行這個函式,並把 s 和 r 的值存入 colors 和 rank
void display() const { //顯示撲克牌的資訊
cout << rank << " of " << colors << endl;
}
};
// 實作 Stack
class Stack {
private:
//使用 vector 作為存放 card 的底層容器
vector<Card> stack; //表示 stack 是一個能存放 Card 類別物件的 vector
public:
//push:將一張牌壓入堆疊的最上方
void push(const Card& card) {
stack.push_back(card); //在 vector 的最後面加入元素，模擬元素在堆疊的最頂端
}
//pop:從堆疊的最上方取出一張牌
Card pop() {
Card topCard = stack.back(); //取得 vector 最後一個元素(堆疊頂端的牌)
stack.pop_back(); //從 vector 中移除該元素
return topCard; //回傳被取出的這張牌
}
//isEmpty:檢查堆疊是否為空
bool isEmpty() const {
return stack.empty(); //若 vector 的長度為0，代表堆疊是空的
}
};
// 代表一副撲克牌的類別
class Deck {
private:
vector<Card> cards; // 存放未洗牌的撲克牌
Stack shuffledDeck; // 存放洗過的牌,用實作的 stack 來管理
input
Shuffled deck:
Q of Diamonds
4 of Clubs
Q of Spades
8 of Clubs
Q of Clubs
5 of Diamonds
3 of Diamonds
K of Clubs
A of Clubs
3 of Spades
8 of Hearts
10 of Hearts
K of Hearts
5 of Spades
K of Spades
4 of Spades
J of Clubs
J of Hearts
5 of Clubs
5 of Hearts
A of Diamonds
8 of Diamonds
3 of Hearts
9 of Spades
2 of Clubs
4 of Diamonds
J of Spades
9 of Diamonds
6 of Clubs
6 of Spades
7 of Diamonds
3 of Clubs
8 of Spades
2 of Diamonds
4 of Hearts
7 of Diamonds
7 of Hearts
2 of Spades
7 of Spades
10 of Diamonds
9 of Clubs
K of Diamonds
6 of Hearts
A of Spades
10 of Clubs
7 of Clubs
6 of Diamonds
9 of Hearts
Q of Hearts
2 of Hearts
10 of Spades
A of Hearts
...Program finished with exit code 0
Press ENTER to exit console.


onlinegdb.com/online_c++_compiler#
Run
Debug
Stop
Share
Save
{ } Beautify
Language C++
main.cpp
#include <iostream>
#include <vector> //用於動態陣列
#include <algorithm> //用於 std shuffle
#include <random> //用於產生隨機數種子
#include <string> //用於處理字串
using namespace std;
// 表示單張撲克牌的類別
class Card {
public:
string colors; //儲存撲克牌的花色
string rank; //儲存撲克牌的數值
Card(string s, string r) : colors(s), rank(r) {} //建立 constructor來初始化物件,當 Card 物件建立時,它會自動執行這個函式,並把 s 和 r
void display() const { //顯示撲克牌的資訊
cout << rank << " of " << colors << endl;
}
};
// 實作 Stack
class Stack {
private:
//使用 vector 作為存放 card 的底層容器
vector<Card> stack; //表示 stack 是一個能存放 Card 類別物件的 vector
public:
//push:將一張牌壓入堆疊的最上方
void push(const Card& card) {
stack.push_back(card); //在 vector 的最後面加入元素，模擬元素在堆疊的最頂端
}
//pop:從堆疊的最上方取出一張牌
Card pop() {
Card topCard = stack.back(); //取得 vector 最後一個元素(堆疊頂端的牌)
stack.pop_back(); //從 vector 中移除該元素
return topCard; //回傳被取出的這張牌
}
//isEmpty:檢查堆疊是否為空
bool isEmpty() const {
return stack.empty(); //若 vector 的長度為0，代表堆疊是空的
}
};
// 代表一副撲克牌的類別
class Deck {
private:
vector<Card> cards; // 存放未洗牌的撲克牌
Stack shuffledDeck; // 存放洗過的牌,用實作的 stack 來管理
public:
Deck() { //建立 constructor 來初始化物件
string colors[] = { "Hearts", "Diamonds", "Clubs", "Spades" };
//儲存撲克牌的花色
input
Shuffled deck:
2 of Clubs
10 of Spades
7 of Spades
9 of Spades
10 of Hearts
2 of Hearts
2 of Spades
3 of Spades
Q of Hearts
5 of Hearts
J of Diamonds
5 of Clubs
J of Hearts
4 of Hearts
7 of Clubs
2 of Diamonds
5 of Spades
K of Spades
8 of Hearts
6 of Diamonds
6 of Spades
Q of Spades
A of Hearts
3 of Diamonds
A of Spades
8 of Spades
A of Clubs
K of Clubs
K of Hearts
6 of Clubs
J of Clubs
4 of Spades
9 of Diamonds
K of Diamonds
4 of Clubs
7 of Diamonds
4 of Diamonds
9 of Clubs
5 of Diamonds
J of Spades
9 of Hearts
Q of Diamonds
3 of Clubs
10 of Diamonds
3 of Hearts
8 of Diamonds
10 of Clubs
A of Diamonds
8 of Clubs
7 of Hearts
Q of Clubs
6 of Hearts
...Program finished with exit code 0
Press ENTER to exit console.

onlinegdb.com/online_c++_compiler#

Run | Debug | Stop | Share | Save | { } Beautify | Language C++

main.cpp

```cpp
#include <iostream>
#include <vector> //用於動態陣列
#include <algorithm> //用於 std shuffle
#include <random> //用於產生隨機數種子
#include <string> //用於處理字串

using namespace std;

// 表示單張撲克牌的類別
class Card {
 public:
 string colors; //儲存撲克牌的花色
 string rank; //儲存撲克牌的數值
 Card(string s, string r) : colors(s), rank(r) {} //建立 constructor來初始化物件,當 Card 物件建立時,它會自動執行這個函式,並把 s 和 r 
 void display() const { //顯示撲克牌的資訊
  cout << rank << " of " << colors << endl;
 }
};

// 實作 Stack
class Stack {
 private:
 //使用 vector 作為存放 card 的底層容器
 vector<Card> stack; //表示 stack 是一個能存放 Card 類別物件的 vector

 public:
 //push:將一張牌壓入堆疊的最上方
 void push(const Card& card) {
  stack.push_back(card); //在 vector 的最後面加入元素，模擬元素在堆疊的最頂端
 }

 //pop:從堆疊的最上方取出一張牌
 Card pop() {
  Card topCard = stack.back(); //取得 vector 最後一個元素(堆疊頂端的牌)
  stack.pop_back(); //從 vector 中移除該元素
  return topCard; //回傳被取出的這張牌
 }

 //isEmpty:檢查堆疊是否為空
 bool isEmpty() const {
  return stack.empty(); //若 vector 的長度為0，代表堆疊是空的
 }
};

// 代表一副撲克牌的類別
class Deck {
 private:
 vector<Card> cards; // 存放未洗牌的撲克牌
 Stack shuffledDeck; // 存放洗過的牌,用實作的 stack 來管理

 public:
 Deck() { //建立 constructor 來初始化物件
  string colors[] = { "Hearts", "Diamonds", "Clubs", "Spades" };
  //儲存撲克牌的花色
```

input

```
Shuffled Deck:
3 of Spades
Q of Clubs
6 of Clubs
J of Spades
A of Clubs
5 of Clubs
10 of Hearts
Q of Diamonds
J of Hearts
A of Diamonds
A of Hearts
10 of Clubs
J of Clubs
4 of Hearts
3 of Diamonds
6 of Hearts
4 of Spades
10 of Spades
8 of Hearts
7 of Diamonds
5 of Hearts
K of Spades
2 of Hearts
2 of Spades
5 of Diamonds
8 of Diamonds
8 of Spades
7 of Clubs
9 of Clubs
2 of Clubs
J of Diamonds
4 of Clubs
Q of Spades
8 of Clubs
2 of Diamonds
7 of Spades
4 of Diamonds
K of Hearts
K of Diamonds
A of Spades
9 of Spades
9 of Diamonds
7 of Hearts
6 of Diamonds
3 of Clubs
Q of Hearts
5 of Spades
3 of Hearts
6 of Spades
K of Clubs
9 of Hearts
10 of Diamonds


...Program finished with exit code 0
Press ENTER to exit console.
```